



**3D Force Button Sensor
Python
(Beta v1.0)**

Installation and Operation Manual

Document #: 3DFBSPython_B1.0_MAN_FEB25

February, 2025

Foreword

Information contained in this document is the property of Contactile Pty Ltd. and shall not be reproduced in whole or in part without prior written approval of Contactile Pty Ltd. The information herein is subject to change without notice and should not be construed as a commitment on Contactile Pty Ltd. This manual is periodically revised to reflect and incorporate changes made to the 3D Force Button Sensor Development Kit.

Contactile Pty Ltd assumes no responsibility for any errors or omissions in this document. Users' critical evaluation is welcome to assist in the preparation of future documentation.

Copyright © by Contactile Pty Ltd, Sydney, Australia. All Rights Reserved.
Published in Australia.

All trademarks belong to their respective owners.

Conditions of Sale

Contactile's conditions of sale apply to all products sold by Contactile to the Distributor under this Agreement. The conditions of sale that apply are provided on the USB flash drive shipped with the product in the folder 'LEGAL' in the root directory.

End User Licence Agreement

Contactile's end user license agreement applies to all software and algorithms included with the products sold by Contactile. The end user license agreement that applies is provided on the USB flash drive shipped with the product in the folder 'LEGAL' in the root directory.

Compliance

The devices are sold as is.

The devices are specifically designed solely for the purposes of research and development only made available on a business-to-business basis.

The devices are not for resale.

Table of Contents

1	Introduction	4
2	Safety	5
2.1	General	5
2.2	Explanation of warnings.....	5
2.3	Precautions	5
3	Getting started	6
3.1	Hardware installation	6
3.2	Software installation	6
3.3	Importing the library	6
4	Class and function documentation	7
4.1	PTSDKConstants class constants.....	7
4.2	PTSDKSensor class public functions	7
4.3	PTSDKListener class public functions	7
5	Writing a user application using the Python Library	9
5.1	Importing the Python Library	9
5.2	Initialising PTSDKSensor and PTSDKListener objects	9
5.3	Connecting to the COM port and listening for data	10
5.4	Biasing the sensors	11
5.5	Accessing sensor data.....	11
6	Log file.....	12
6.1	Overview.....	12
6.2	Log file location	12
6.3	Log file name.....	12
6.4	Log file format	12

1 Introduction

The 3D Force Button Sensor Development Kit (Beta v1.0) is a system of (up to) five 3D Force Button Sensors per adapter, and a DEV001 development board. Each 3D Force Button Sensor can measure 3D force. The DEV001 development board supplies power for, and coordinates the simultaneous data acquisition from, up to five 3D Force Button Sensors. The DEV001 development board is shipped with visualisation software, C++ and Python libraries for Windows and Linux environments and ROS and ROS2 nodes for developing software control algorithms using the sensor signals.

Two 3D Force Button Sensors are shown in Figure 1.1, connected to a DEV001 development Board with USB-C cable.



Figure 1.1 – Two 3D Force Button Sensors connected to a DEV001 development board and USB-C cable.

This document is an installation and operation manual for the Python Library which was provided on the Contactile USB flash drive that was shipped with the 3D Force Button Sensor and DEV001 development board.

2 Safety

2.1 General

The customer should verify that the maximum loads and moments expected during operation fall within the sensing range of the sensor as outside this range, sensor reading accuracy is not guaranteed (refer to Document #3DFBS_Datasheet_RevX.X). Particular attention should be paid to dynamic loads caused by robot acceleration and deceleration if the sensors are mounted on robotic equipment. These forces can be many multiples of the value of static forces in high acceleration or deceleration situations.

2.2 Explanation of warnings

The warnings included here are specific to the product(s) covered by this manual. It is expected that the user heed all warnings from the manufacturers of other components used in the installation.



Danger indicates that a situation could result in potentially serious injury or damage to equipment.



Caution indicates that a situation could result in damage to the product and/or the other system components.

2.3 Precautions



DANGER: Do not attempt to disassemble the sensor. This could damage the sensor and will invalidate the calibration.



DANGER: Do not attempt to drill, tap, machine, or otherwise modify the sensor casing. This could damage the sensor and will void invalidated the calibration.



DANGER: Do not use the sensor on abrasive surfaces or surfaces with sharp points/edges. This could damage the silicone surface of the sensor.



CAUTION: Sensors may exhibit a small offset in readings when exposed to intense light sources.



CAUTION: Exceptionally strong and changing electromagnetic fields, such as those produced by magnetic resonance imaging (MRI) machines, constitute a possible source of interference with the operation of the sensor and Controller.



CAUTION: Temperature variations can cause drift in sensor readings. Some temperature compensation is performed. However, bias removal in software prior to operation is necessary, and it is recommended that biasing is performed each time the sensor is known to be unloaded.

3 Getting started

This section contains instructions for setting up and using 3D Force Button Sensor Python Library for WINDOWS (Beta v1.0). It is recommended that first time users first read the preceding Safety section, then read through this section to get more familiar with the system.

3.1 Hardware installation

The Python Library is used with the 3D Force Button Sensor and DEV001 development board. The DEV001 development board should be connected to the 3D Force Button Sensors, then the DEV001 development board should be connected via the USB-C port on the DEV001 development board to a computer running VSCode (in Windows or Linux) before you can use the Python Library. For more information about connecting the sensors and powering on the DEV001 development board, refer to Document DEV001_Datasheet_RevX.X.

3.2 Software installation

The Python Library is provided on the Contactile USB flash drive that was shipped with the development kit in the folders named *SOFTWARE/PythonLIN* and *SOFTWARE/PythonWIN* for Linux and Windows, respectively.

The files in the *PythonLIN* and *PythonWIN* folders are summarised in Table 3.1.

Table 3.1 – Files in *PythonLIN* and *PythonWIN* folders

Sub Folder	File Name	File Description
PythonLIN	fbs3d_cxx_pybind-1.0.0-cp310-cp310-linux_x86_64.whl	The Python wheel file for Linux
	fbs3d_pybind_example.py	The example Python code for Linux
PythonWIN	fbs3d_cxx_pybind-1.0.0-cp310-cp310-win_amd64.whl	The Python wheel file for Windows
	fbs3d_pybind_example.py	The example Python code for Windows

To install the library, in a terminal window in VSCode, navigate to the folder containing the relevant wheel file.

In Windows call: `pip install name_of_wheel_file.whl`
In Linux, call: `pip3 install name_of_wheel_file.whl`

3.3 Importing the library

In your Python code include the following: `import FBS3D_CXX_Pybind`

4 Class and function documentation

In this section, the classes and class functions of the Python Library are described.

4.1 PTSDKConstants class constants

The PTSDKConstants class contains definitions of constants that can be used in the user application. The constants are described in Table 4.1.

Table 4.1 – Constants defined the class PTSDKConstants

Name	Value	Description
X_IND	0	The index of the X-dimension
Y_IND	1	The index of the Y-dimension
Z_IND	2	The index of the Z-dimension
NDIM	3	The number of spatial dimensions
LOG_RATE_100	100	Constant representing 100 Hz sampling rate
LOG_RATE_500	500	Constant representing 500 Hz sampling rate
LOG_RATE_1000	1000	Constant representing 1000 Hz sampling rate

4.2 PTSDKSensor class public functions

The PTSDKSensor is a class that describes a 3D Force Button Sensor (v2.0). This is the main class for accessing the current sensor measurements in a user-defined program.

__init__(self)

Description: Constructor

def getGlobalForce(self)

Description: Gets the global X,Y,Z force acting on the sensor.

Returns: The global X, Y and Z force. A list with three elements.

def getTimestamp_us(self) -> int

Description: Gets the timestamp of the current sample in μ s.

Returns The timestamp of the current sample in us.

4.3 PTSDKListener class public functions

The PTSDKListener is the class which interacts with the DEV001 development board that is in turn hosting up to five connected 3D Force Button Sensors. This class describes an object that connects with the DEV001 development board via a serial connection emulated on the computer's USB port, and reads and processes the streamed data. This class also logs the data to a log file – See Section 6 Log file. The member functions of the PTSDKListener class are described below.

def __init__(self, logFlag: bool) -> None

Description: Constructor.

Parameters: self

logFlag A flag indicating whether to log data to CSV file.

def addSensor(self, pSensor: PTSDKSensor) -> None

Description: Adds a sensor object to the PTSDKListener.

Parameters: self
pSensor A sensor object.

def connect(self, port: str, rate: int, parity: int, byteSize: str, logFileRate: int) -> int

Description: Connects to the COM port.

Used in conjunction with the readNextSample and disconnect functions.

Parameters: self
port The COM port name.
rate The rate of the connection.
parity The parity of the connection.
byteSize The byte size. A single Unicode character – e.g. ‘\u0008’

Returns: 0 if successfully connected, error code if unsuccessful.

def startListening(self) -> None

Description: Starts listening for data (starts the listening thread), processes the data and logs the data to a log file. Can only be called after connect.

Used in conjunction with the stopListeningAndDisconnect function.

Parameters: self

def connectAndStartListening(self, port: string, rate: int, parity: int, byteSize: str, logFileRate: int) -> int

Description: Connects to the COM port and starts listening for data (starts the listening thread), processes the data and logs the data to a log file.

Used in conjunction with the stopListeningAndDisconnect function.

Parameters: self
port The COM port name.
Rate The rate of the connection.
Parity The parity of the connection.
byteSize The byte size. A single Unicode character – e.g. ‘\u0008’
logFileRate The log file rate. LOG_RATE_100, LOG_RATE_500 Hz, or LOG_RATE_1000 for 100, 500 or 1000 Hz, respectively.

Returns: 0 if successfully connected, error code if unsuccessful.

def disconnect(self) -> None

Description: Disconnects from the COM port.

Used in conjunction with the connect and readNextSample functions.

def stopListeningAndDisconnect(self) -> None

Description: Stops listening for data from the COM port (and kills the listening thread), stops logging data to the log file and disconnects from the COM port.

def sendBiasRequest(self)-> bool

Description: Sends a bias request to all sensors connected to the DEV001 development board. A bias should be performed after connecting to the serial port and starting to stream data with the sensor unloaded. A bias should be performed each time the sensor is known to be unloaded. A bias operation takes approximately 2 s. Ensure that the sensor remains unloaded throughout this time.

Returns: True if successfully sent the request, false if unsuccessful.

5 Writing a user application using the Python Library

This section contains code snippets explaining the steps required to write a Python user application to monitor ten 3D Force Button Sensors. The full example can be found in the `fbs3d_pybind_example.py` file in the *PythonXXX* folder (where XXX is LIN or WIN for Linux and Windows, respectively).

5.1 Importing the Python Library

These examples require the imports listed in Example 5.1.

Example 5.1 – Imports for the example user application

```
import FBS3D_CXX_Pybind
```

5.2 Initialising PTSDKSensor and PTSDKListener objects

To initialise a PTSDKListener object, first, the PTSDKSensor objects must be initialised. Ten sensors should be initialised irrespective of how many physical sensors are connected. An example of initialising ten PTSDKSensor objects then initialising the PTSDKListener object is shown in Example 5.2.

NB: even though, only five 3DFB Sensors can be connected to the DEV001 development board, ten sensor objects are required to be initialised and added to the listener object for backwards compatibility with the older analogue 3D Force Button Sensor.

Example 5.2 – Initialising two PTSDKSensor objects and a PTSDKListener object

```
# Initialise 10x sensors irrespective of number of physical sensors
sen0 = FBS3D_CXX_Pybind.PTSDKSensor()
sen1 = FBS3D_CXX_Pybind.PTSDKSensor()
sen2 = FBS3D_CXX_Pybind.PTSDKSensor()
sen3 = FBS3D_CXX_Pybind.PTSDKSensor()
sen4 = FBS3D_CXX_Pybind.PTSDKSensor()
sen5 = FBS3D_CXX_Pybind.PTSDKSensor()
sen6 = FBS3D_CXX_Pybind.PTSDKSensor()
sen7 = FBS3D_CXX_Pybind.PTSDKSensor()
sen8 = FBS3D_CXX_Pybind.PTSDKSensor()
sen9 = FBS3D_CXX_Pybind.PTSDKSensor()

# Initialise listener and add 10x sensors irrespective of number of physical sensors
isLogging = True
listener = FBS3D_CXX_Pybind.PTSDKListener(isLogging)
listener.addSensor(sen0)
listener.addSensor(sen1)
listener.addSensor(sen2)
listener.addSensor(sen3)
listener.addSensor(sen4)
listener.addSensor(sen5)
listener.addSensor(sen6)
listener.addSensor(sen7)
listener.addSensor(sen8)
listener.addSensor(sen9)
```

5.3 Connecting to the COM port and listening for data

After initialising the PTSDKListener, a serial connection must be established.

Note: There should be a COM port associated with the DEV001 development board. To power the board, the USB cable should be connected between the USB-C port on the board and the computer.

The COM port configuration parameters are first required. An example of initialising the COM port configuration parameters is shown in Example 5.3.

Example 5.3 – COM port configuration parameter initialisation

```
# Connection parameters
port = .\\\\.COM3"    # The name of the COM port to connect with
                    # For Windows, the additional '\' characters are required
                    # For Linux, this could be "/dev/ttyACM0"
rate = 115200        # The rate of the serial connection
parity = 0           # 0=PARITY_NONE, 1=PARITY_ODD, 2=PARITY_EVEN
byteSize = '\u0008'  # The number of bits in a byte, as a single Unicode character
```

After the PTSDKListener has established a connection with the COM port, the DEV001 development board must be sent a bias command to begin streaming data through the serial connection. The PTSDKListener object can launch a thread which listens for incoming data packets. An example of connecting to the COM port and listening for data is shown in Example 5.4.

Example 5.4 – Connecting to the COM port and listening for data in a multi-threaded application

```
# Connect to COM port
res = listener.connect(port, rate, parity, byteSize)
if res == 0:
    print('main(): Successfully connected to ' + port + ' and started listening')
else:
    print('main(): Failed to connect to ' + port)
    exit()

# Send bias to start data streaming
res = listener.sendBiasRequest()
if res:
    print('main(): Successfully biased the sensors')
else:
    print('main(): Failed to bias the sensors')
    exit()

# Start listening - multi-threaded
listener.startListening()

# User application code - do something with the data
while True:
    pass

# Stop listening and disconnect from COM port
listener.stopListeningAndDisconnect()
```

5.4 Biasing the sensors

Biasing refers to removing any offset in the pillar readings when the pillars are unloaded. It is recommended that the user performs a bias each time the sensors are known to be unloaded. Ensure that the sensor has been unloaded for at least one second before performing a bias to ensure that the bias calculation does not include hysteresis effects. A bias operation can take up to 2 s. Ensure that the sensor remains unloaded throughout this time. An example of how to perform a bias is shown in Example 5.5.

Example 5.5 – Biasing all pillars on all sensors

```
# Send bias
res = listener.sendBiasRequest()
if res == 0:
    print('main(): Successfully biased the sensors')
else:
    print('main(): Failed to bias the sensors')
    exit()
```

5.5 Accessing sensor data

Once the PTSDKListener object is listening for data and the sensors have been biased, the user application can access the incoming sensor data. An example of how to access data from a sensor is shown in Example 5.6.

Example 5.6 – Accessing data from a sensor

```
force = sen0.getGlobalForce()
for dimInd in range(0, FBS3D_CXX_Pybind.PTSDKConstants.NDIM):
    print('S0: global F' + str(dimInd) + ' = ' + str(force[dimInd]))
```

6 Log file

6.1 Overview

If the PTSDKListener object is initialised with the isLogging flag being true, the function connectAndStartListening (in a multi-threaded application) also generates a log file of the data.

6.2 Log file location

The log file that is generated is stored in the Logs subfolder in the same location as the user-defined application which uses the Python Library.

6.3 Log file name

The name of the log file that is generated is LOG_YYYY_MM_DD_hh_mm_ss.csv where:

- YYYY is the four digit year,
- MM is the two digit month,
- DD is the two digit day,
- hh is the two digit hour,
- mm is the two digit minute and
- ss is the two digit second,

from the system clock at the time that the log file was created.

6.4 Log file format

The log file is saved as comma-separated values (CSV) in ASCII text format. The order of the values and a description is shown in Table 6.1.

Table 6.1 – Data in log file

Data Order	Data Name	Data Description	
1	T_us	Timestamp in μ s	Timestamps
2	S0_G_FX	Sensor 0, X-axis force in Newtons	Sensor 0, forces
3	S0_G_FY	Sensor 0, Y-axis force in Newtons	
4	S0_G_FZ	Sensor 0, Z-axis force in Newtons	
⋮			
29	S9_G_FX	Sensor 9, X-axis force in Newtons	Sensor 9, forces
30	S9_G_FY	Sensor 9, Y-axis force in Newtons	
31	S9_G_FZ	Sensor 9, Z-axis force in Newtons	

S0, refers to the sensor connected to PORT 0, S1 refers to the sensor connected to PORT 1, and so on, and S4 refers to the sensor connected to PORT 4 of the DEV0001 development.

NB: S5 to S9 sensors will have all data equal to zero – these are necessary for backward compatibility with the older analogue 3D Force Button Sensor.

If a sensor is not connected, the log file will contain values of 0.0 for FX, FZ and FZ in the corresponding data columns.